

澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

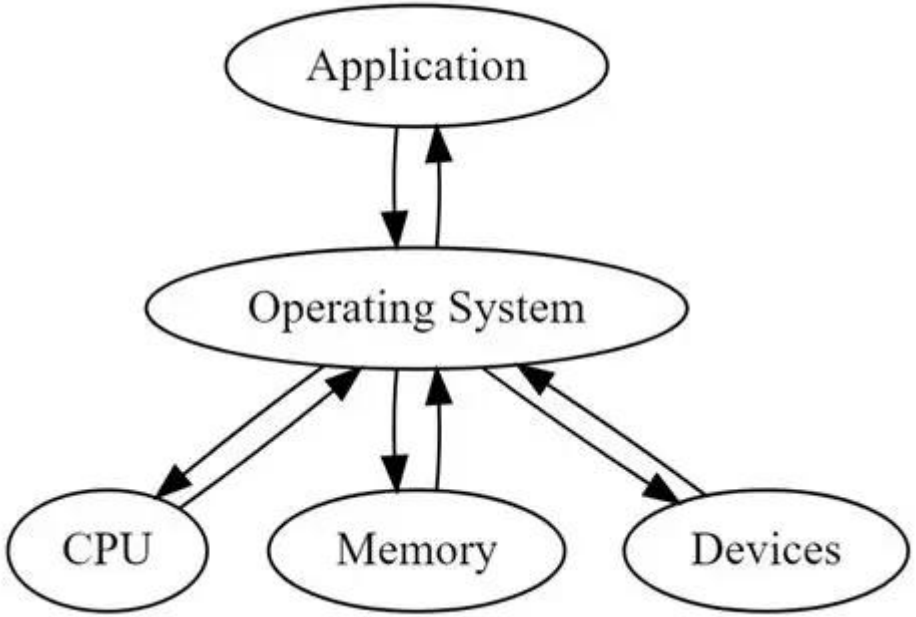
COMP2113 — Operating Systems

Process Description and Control

WEEK-02 | Reference: Chapter 3

OS

We can think of the OS as providing a uniform, abstract representation of resources that can be requested and accessed by applications. Resources include main memory, network interfaces, file systems, and so on. Once the OS has created these resource abstractions for applications to use, it must also manage their use. For example, an OS may permit resource sharing and resource protection.



Process Description and Control

- Why process?
- What is a process?
- How to represent processes?
- How to control processes?



What is a Process?

- "A program in execution", also called a **task**
- An instance of a program running on a computer
- The entity that can be assigned to and executed on a processor
- A unit of activity characterized by a sequence of instructions, a current state, and associated system instructions

Note the difference between program and process.

Process Elements

A process is comprised of:

Program Code

The instructions to be executed. Possibly shared among multiple processes.

Data

A set of data used and modified by the process during execution.

Attributes

State information describing the process, maintained by the OS.

Process Elements While Running

While the process is running, it has a number of elements including:

- **Identifier** — unique process ID
- **State** — current execution state
- **Priority** — scheduling priority
- **Program Counter** — address of next instruction

- **Memory Pointers** — pointers to code & data
- **Context Data** — register contents
- **I/O Status** — open files, devices
- **Accounting Info** — CPU time, limits

Process Control Block (PCB)

PCB contains data that the OS needs to **control and manage** a process.

Process Identification

- pid: 123
- uid: tom
- Parent process ID

Processor State Info

- eax, ebx, ecx, ...
- eflags (status flags)
- eip (instruction pointer)
- cs, ds, ss (segments)

Process Control Info

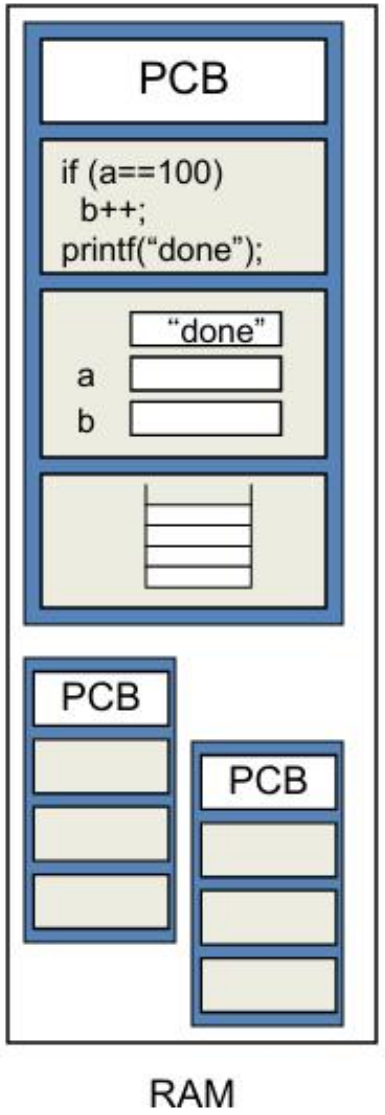
- Scheduling state & priority
- Memory management info
- I/O status & open files
- Accounting information

The Process Image

OS maintains the following for each process:

- **Process Control Block (PCB)**
- **User Program** (.EXE, .DLL, etc.)
- **User Data** (variables, constants, buffers)
- **User Stack** (local vars, call addresses)

RAM Layout



Requirements in Process Management

Interleave the execution of several processes

— to maximize processor utilization | while providing reasonable response time

Allocate resources to processes

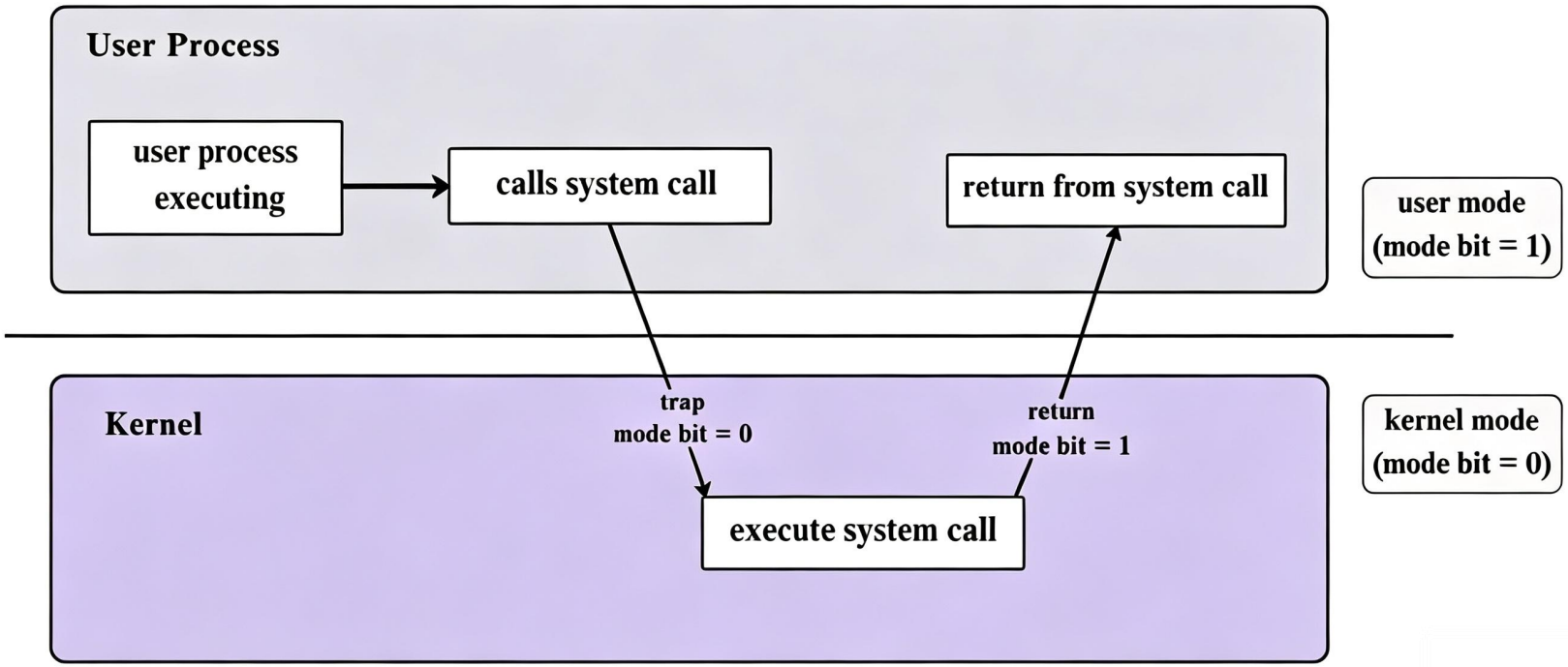
Support interprocess communication

User Mode vs Kernel Mode

- User Mode: Normal user program runs here, **limited** privilege
- Kernel Mode (System Mode): OS core runs here, **full** privilege

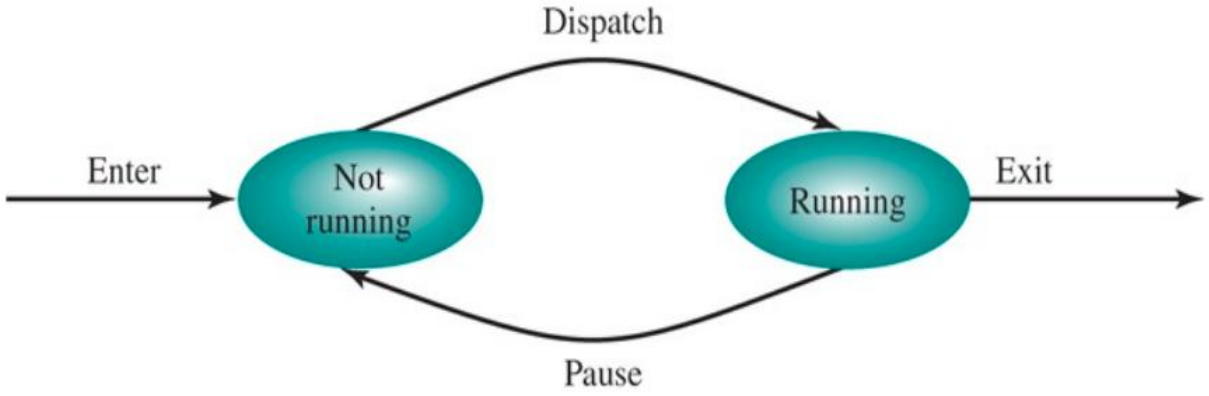
- Process control operations MUST run in Kernel Mode
- Trigger ways: System call / Interrupt

User Mode And Kernel Mode



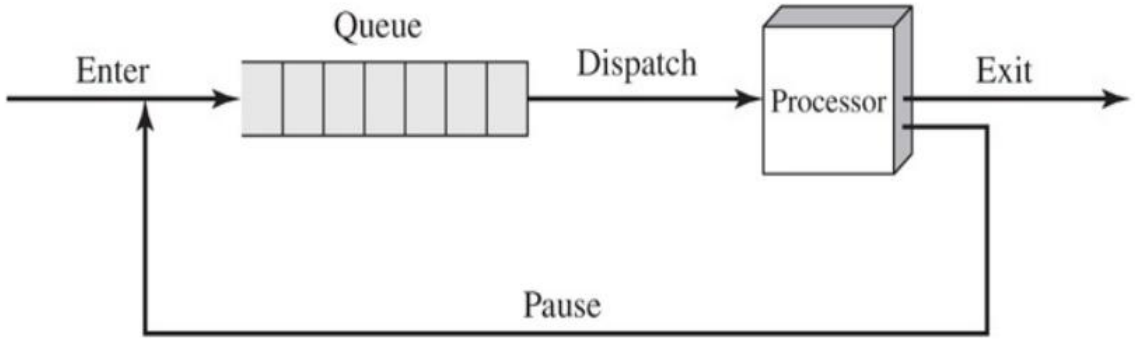
Transition from user to kernel mode

A Two-State Process Model



(a) State transition diagram

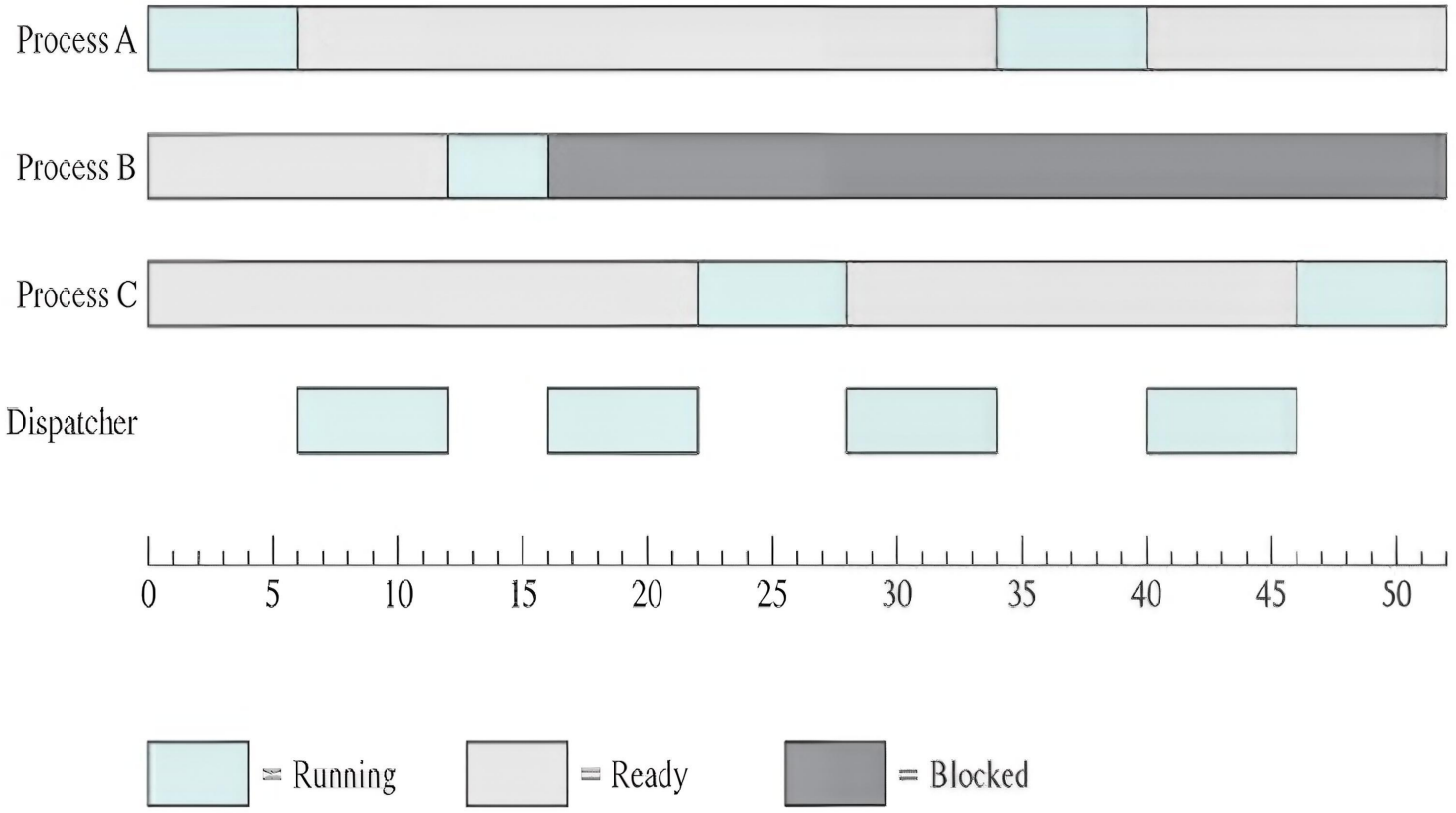
The process exists, is known to the OS, and is waiting for an opportunity to execute. From time to time, the currently running process will be interrupted, and the dispatcher portion of the OS will select some other process to run. The former process moves from the Running state to the Not Running state, and one of the other processes moves to the Running state.



(b) Queueing diagram

We can describe the behavior of the dispatcher in terms of this queueing diagram. A process that is interrupted is transferred to the queue of waiting processes. Alternatively, if the process has completed or aborted, it is discarded (exits the system). In either case,the dispatcher takes another process from the queue to execute.

Transition of each process among the states



This diagram illustrates a CPU scheduling timeline in an operating system. It shows three processes, A, B, and C, over time, along with the actions of the dispatcher. Each process can be in one of three states: Running, Ready, or Blocked.

Process Creation

- **Submission of a batch job**
- — OS creates process for queued job
- **User logs on**
- — interactive session creates a process
- **Created to provide a service**
- — e.g., printing, network daemon
- **Spawned by an existing process**
- — parent creates child process

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

Process Termination (1/2)

- **Batch job issues Halt instruction** — normal completion
- **User logs off** — interactive session ends
- **Process executes a service request to terminate** — voluntary exit
- **Parent terminates** — child processes may also terminate

Process Termination (2/2)

Operating System Intervention

— E.g., when deadlock occurs, the OS may terminate a process to break the deadlock.

Error and Fault Conditions

— E.g., memory unavailable, protection error, arithmetic error, I/O failure, invalid instruction.

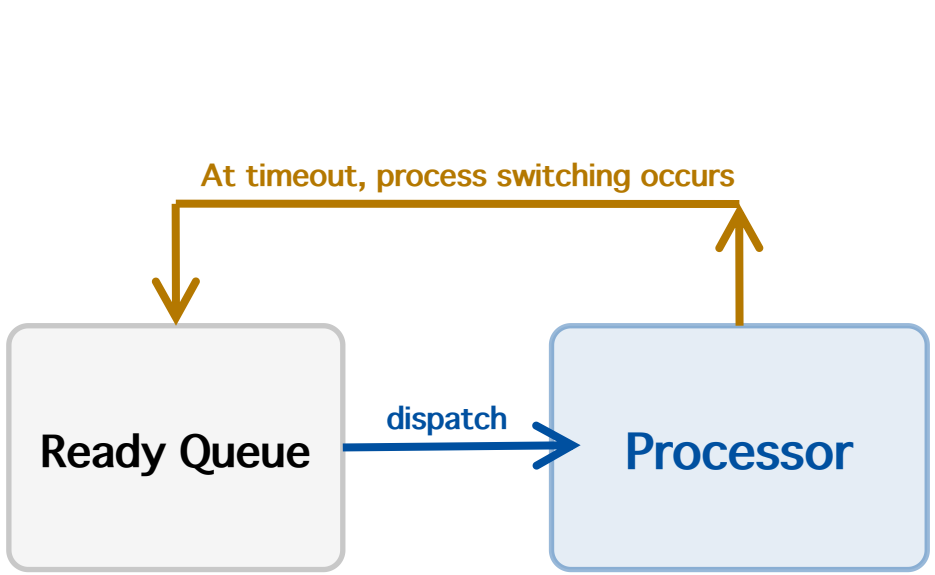
The Dispatcher

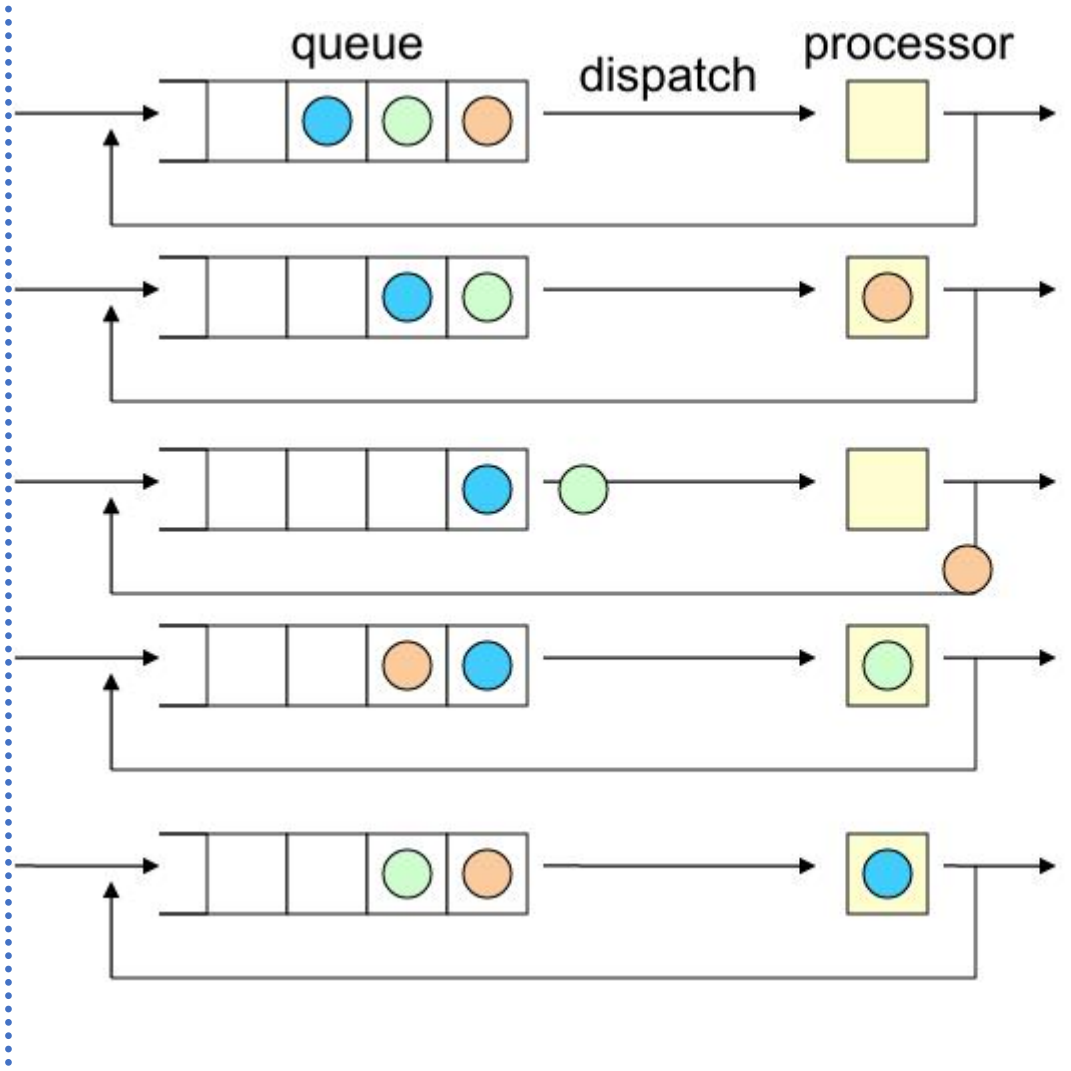
A **small program** that switches the processor from one process to another.

Prevents a single process from monopolizing processor time

— Every process should be able to use the processor for a fair amount of time.

Round-Robin Scheduling





Each process is allowed to execute for at most a quantum.

At timeout, process switching occurs.

When is a Process Preempted?

In the following two cases, a process cannot continue running and must leave the processor — i.e., it is **preempted**.

Timeout

The process has used its time quantum and must yield the processor.

I/O or Event Wait

The process needs to wait for I/O completion or another event.

We will learn more cases of process switching later in this chapter.

Five-State Model

Running — being executed by the processor

Ready — ready to execute, waiting for processor

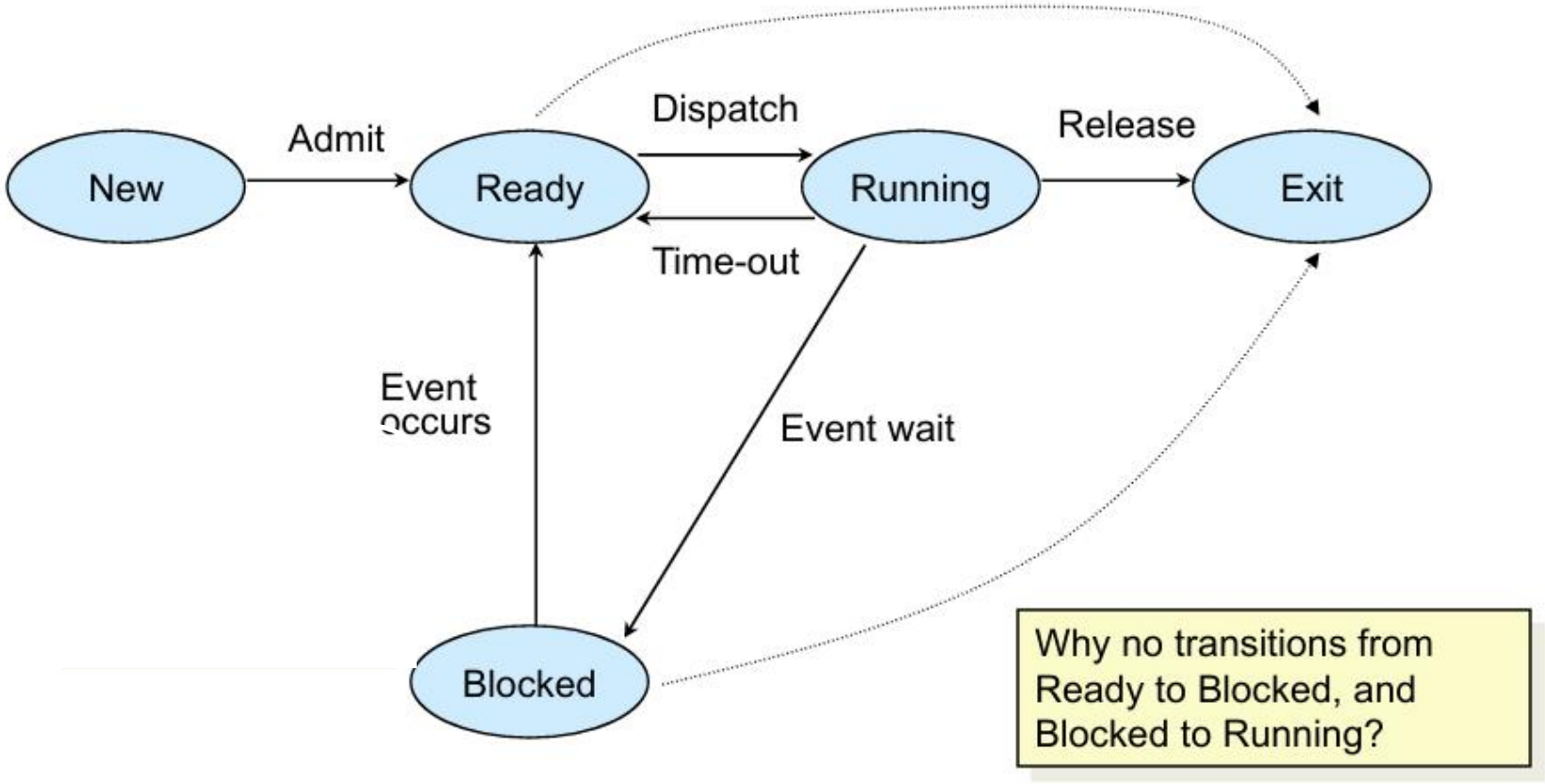
Think about the order

Blocked — cannot execute until some event occurs

New — created, but not yet admitted to the ready queue

Exit — released, no longer eligible for execution

Five-State Process Model



Think: Why no transitions from Ready to Blocked, and Blocked to Running?

Priority Scheduling

Note: more scheduling policies in Chapter 9

- Each process is assigned a **priority**
- One ready queue for the ready processes of a certain priority
- When the OS dispatches, it chooses a ready process of the **highest priority**
- When a high priority process becomes ready, it may **preempt** a running process of lower priority

Suspend Processes

Processor is faster than I/O, so all processes could be waiting for I/O.
The **Blocked** state becomes **Suspend** when swapped to disk.

Suspend = stop execution temporarily

- OS may suspend a process that causes a problem
- Interactive user request (for debugging)
- Timing: a process that executes periodically
- Swapping: OS may swap a blocked process to disk to free memory

Suspend States

Two new states are added:

Ready / Suspend

Ready to run, but swapped out to disk

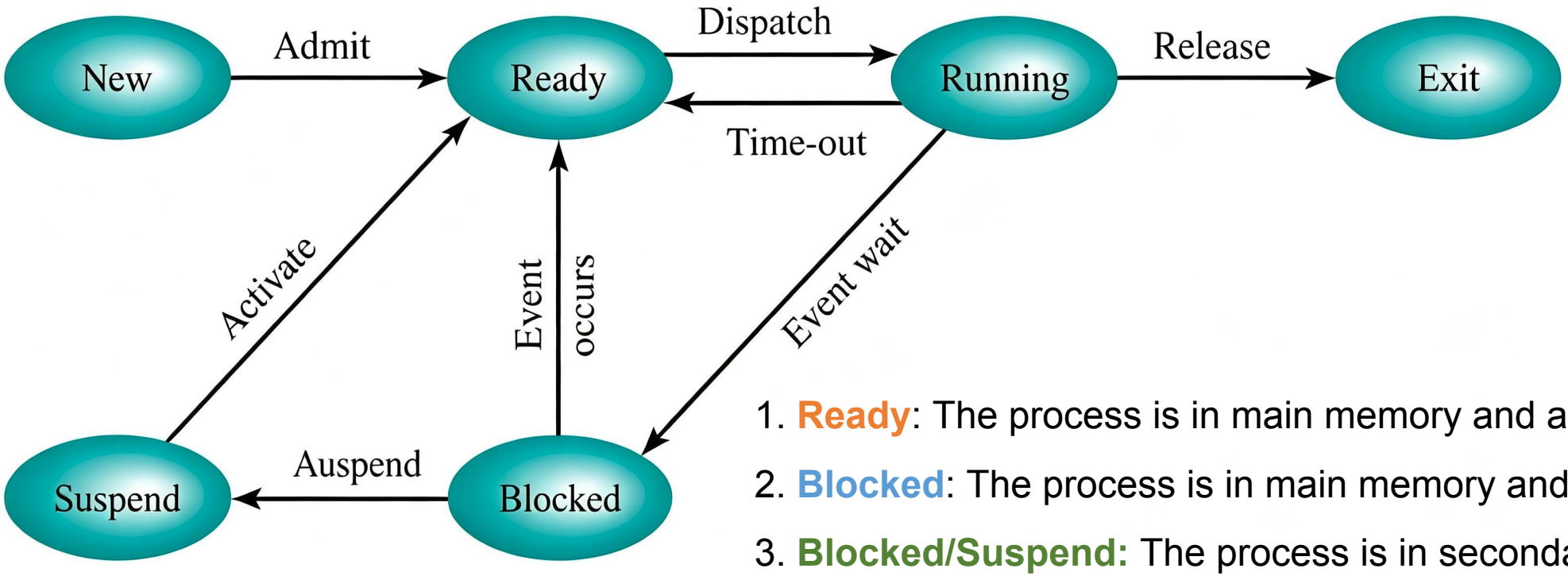
Blocked / Suspend

Waiting for an event, and swapped to disk

Think about these questions:

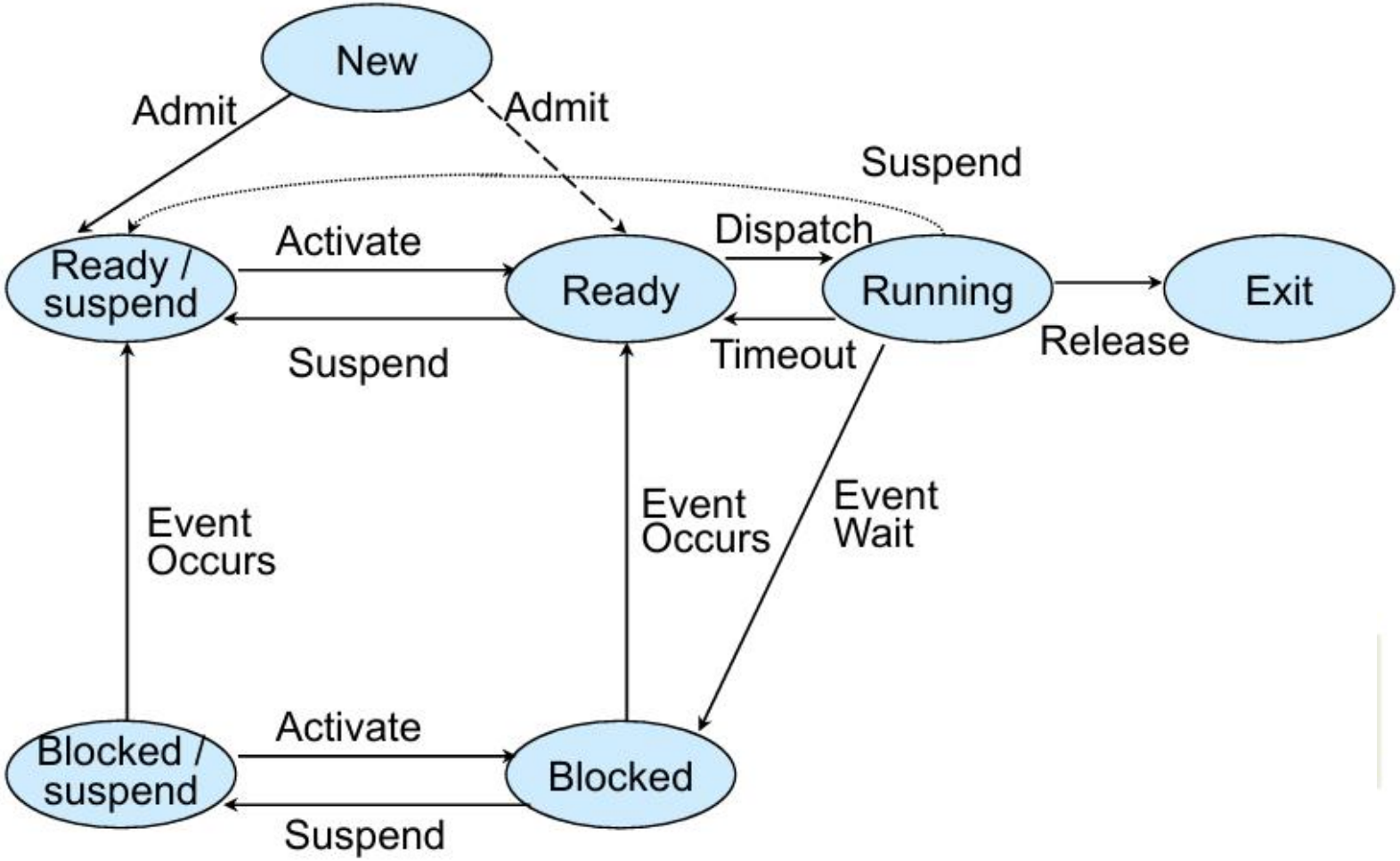
- What's the meaning of the two new states?
- Why not a single 'suspend' state?
- Why no 'running, suspend' state?

State Transitions with One Suspend State



1. **Ready**: The process is in main memory and available for execution.
2. **Blocked**: The process is in main memory and awaiting an event.
3. **Blocked/Suspend:** The process is in secondary memory and awaiting an event.
4. **Ready/Suspend**: The process is in secondary memory but is available for execution as soon as it is loaded into main memory.

State Transitions with Suspend States



Transition Legend

- Admit: New → Ready
- Dispatch: Ready → Running
- Timeout: Running → Ready
- Event Wait: Running → Blocked
- Event Occurs: Blocked → Ready
- Release: Running → Exit
- Suspend: Ready/Blocked → Suspend
- Activate: Suspend → Ready/Blocked

Process Control Primitives

Process Control Primitives

Definition: Atomic kernel functions to change process state

1. Create (Admit)	New → Ready
2. Dispatch	Ready → Running
3. Time-out	Running → Ready (preemption)
4. Block (Event wait)	Running → Blocked
5. Wakeup (Event occurs)	Blocked → Ready
6. Suspend	Blocked/Ready → Suspend
7. Activate	Suspend → Ready
8. Terminate (Release)	Running → Exit

Context Switch

Context Switch
Definition: Switch CPU from one running process to another

Steps:
1. Save context of current running process (registers, PC → PCB)
2. Load context of the next ready process from its PCB
3. Resume execution of new process

Overhead: Pure system cost, no useful work for users
→ Context switch consumes CPU time but does not execute user tasks.

The "Dispatcher" in timeline diagram = context switch
→ The dispatcher performs the context switch operation.

When to Switch Processes

Interrupt

Interruption request from hardware external to the CPU.

- Clock (timer)
- I/O completion
- I/O error

Trap

An error or exceptional condition associated with the current instruction.

- Memory fault
- Division by zero
- Invalid opcode

Supervisor Call

A call to special functions of the OS (system calls).

- File open / read / write
- Synchronization primitives
- Memory allocation

Summary of Process Control

Summary of Process Control

- Process control relies on kernel-mode primitives
- Primitives drive state transitions of processes
- Ready & blocked queues organize waiting processes
- Context switch enables CPU sharing, but brings overhead

Quick Quiz

Q1: Can a user program directly invoke a block primitive?

Q2: What is the difference between Block and Suspend?

Review Questions

1. What is the difference between a program and a process?
2. Name and describe the five states in the Five-state Model.
3. Why is there no transition from Ready to Blocked?
4. What is the difference between Blocked and Blocked/Suspend?
5. What three categories of information does a PCB contain?